

Introduction

If you've spent any time around Python developers, you've heard the claim: "dictionaries are fast." It gets repeated in tutorials, thrown around in interviews, and treated as a fact you're supposed to accept on faith. Almost nobody explains the mechanism behind it.

That's a problem, because interviewers love probing exactly this gap. "Why is dict lookup $O(1)$?" is one of those questions that separates candidates who memorized a fact from candidates who understand a system. It also matters far beyond Python — the same idea powers sets, database indexes, and caching layers across nearly every language and system you'll ever touch. Once you understand hashing, you'll start recognizing "compute the address instead of searching" as a pattern that shows up everywhere in software engineering.

This lesson breaks down exactly what happens when you look something up in a list versus a dictionary, why one scales linearly and the other stays flat, and why Python enforces a strict rule about what can and can't be used as a dictionary key.

Problem Overview

At its core, this isn't a coding problem in the traditional LeetCode sense — it's a systems question: **given a collection of key-value pairs, how do you retrieve a value by its key as fast as possible?**

There are two competing approaches:

1. **Linear search** — store items in a list and check each one until you find a match.
2. **Hash-based lookup** — compute a number from the key that tells you exactly where the value lives, and jump straight there.

Understanding why the second approach wins — and *how* it actually works — is the goal here.

Example

Suppose you're storing user scores:

```
scores = {"alice": 92, "bob": 85, "carol": 78}
```

When you run `scores["bob"]`, Python does not loop through `"alice"`, then `"bob"`, then `"carol"`. It takes the string `"bob"`, runs it through a hash function, gets back a number, uses that number to compute a slot index in an internal table, and reads the value directly from that slot.

Compare that to a list of tuples:

```
scores_list = [("alice", 92), ("bob", 85), ("carol", 78)]
```

To find Bob's score here, Python checks `scores_list[0]`, sees it doesn't match, checks `scores_list[1]`, finds `"bob"`, and stops. With 3 items this is trivial. With 3 million items, that's a lot of checking on the way to a match — or all 3 million checks if the item isn't there at all.

Intuition

Think about how you'd find a coat at a coat-check counter. One approach: the attendant walks down every hook checking labels until they find yours. Another approach: you hand over a ticket with your hook number printed on it, and they walk straight there.

The ticket doesn't describe your coat — it encodes an *address*. That's the entire trick behind a hash table. Instead of searching for where something *is*, you compute where it *must be*, based only on the key.

An experienced engineer arrives at this idea by asking: "what's the actual bottleneck in searching?" The bottleneck is that a list has no relationship between a key's *value* and its *position*. Position is arbitrary — determined only by insertion order. If you could instead derive position directly from the key, you'd never need to search at all. That derivation is exactly what a hash function does: it takes a key and deterministically produces a number, which gets mapped to a slot index in an internal array.

The remaining engineering problem is what happens when two different keys hash to the same slot — a **collision**. Real hash tables handle this with strategies like open addressing (probe to a nearby free slot) or chaining (store a small list at that slot). These add a small constant amount of overhead, but critically, they don't scale with the total size of the dictionary — which is why lookups stay close to instant even as the dictionary grows.

Brute Force Solution

Idea: Store key-value pairs in a list and scan linearly for a match, exactly like searching an unsorted list.

Algorithm: 1. Iterate over each (key, value) pair. 2. Compare the target key against the stored key. 3. Return the value on a match; otherwise continue to the next pair.

Advantages: - Trivial to implement. - No hash function needed — works for any comparable type, including mutable ones like lists.

Disadvantages: - Lookup cost grows linearly with the number of items. - Every miss (searching for a key that doesn't exist) costs a full scan.

Complexity: - Time: $O(n)$ per lookup, worst case. - Space: $O(n)$ to store n pairs.

Optimal Solution

The optimal approach is a **hash table**, which is what Python's `dict` is implemented as under the hood.

Step-by-step:

Step	What happens
1	Key is passed to Python's built-in hash function, <code>hash(key)</code>
2	The hash (a large integer) is reduced to an index within the table's current size (<code>hash % table_size</code> , conceptually)
3	Python checks the slot at that index
4	If empty, the key doesn't exist. If occupied and the stored key matches, return the value
5	If occupied by a <i>different</i> key (collision), probe to another slot using a defined sequence, and repeat step 4

Key "bob" → `hash("bob")` → some large integer → reduced to slot index 4

Table:

```
[0] -----
[1] -----
[2] carol: 78
[3] -----
[4] bob: 85      <-- found directly, no scanning
[5] alice: 92
```

Because computing a hash and indexing into an array are both constant-time operations, the whole lookup is constant-time *on average* — regardless of how many items are in the table.

Python Code

```
class SimpleHashTable:
    """A minimal hash table to illustrate the mechanism dict uses internally."""
```

```
def __init__(self, size=8):
    self.size = size
    self.table = [[] for _ in range(size)] # chaining for collisions

def _index(self, key):
    return hash(key) % self.size

def insert(self, key, value):
    bucket = self.table[self._index(key)]
    for i, (k, _) in enumerate(bucket):
        if k == key:
            bucket[i] = (key, value) # overwrite existing key
            return
    bucket.append((key, value))

def get(self, key):
    bucket = self.table[self._index(key)]
    for k, v in bucket:
        if k == key:
            return v
    raise KeyError(key)
```

Code Walkthrough

- `_index(key)` calls Python's built-in `hash()` on the key and reduces it into a valid slot number using modulo. This is the "compute the address" step.
- `self.table` is a list of buckets rather than a list of single slots — this is **chaining**, one of the standard collision-resolution strategies. Each bucket can hold multiple `(key, value)` pairs that happened to hash to the same slot.
- `insert` computes the target bucket, then checks whether the key already exists in that bucket (to support overwriting), otherwise appends a new pair.
- `get` computes the same bucket and scans only *that bucket* — not the whole table — for a matching key.

Note the key detail: even with collisions, you're scanning a small bucket, not the entire dataset. That's what keeps this fast even in the worst realistic case.

Dry Run

Using `SimpleHashTable(size=8)` :

```
ht = SimpleHashTable()
ht.insert("alice", 92)
```

```
ht.insert("bob", 85)
ht.insert("carol", 78)

ht.get("bob")
```

1. `hash("bob") % 8` → say this evaluates to `4`.
2. `self.table[4]` is checked — it contains `[("bob", 85)]`.
3. The loop in `get` finds `k == "bob"` immediately and returns `85`.

No comparison against `"alice"` or `"carol"` ever happens. Compare this to the brute-force list version, which would have checked `"alice"` first, found no match, then checked `"bob"`.

Complexity Analysis

Operation	Brute force (list)	Hash table (dict)
Lookup (average)	$O(n)$	$O(1)$
Lookup (worst case)	$O(n)$	$O(n)^*$
Insert	$O(1)$ (append) / $O(n)$ (no duplicates check)	$O(1)$ average
Space	$O(n)$	$O(n)$

*Worst case for a hash table occurs only under pathological collision patterns (e.g., every key hashing to the same slot), which is rare in practice and actively mitigated by Python's hash function design and table resizing.

The $O(1)$ average case holds because the number of items examined per lookup doesn't grow with the total size of the table — it only depends on the (typically very small) number of collisions in a single bucket.

Alternative Solutions

- **Balanced binary search tree (e.g., a sorted structure):** gives $O(\log n)$ lookups. Better worst-case guarantees than a naive hash table, but slower on average than proper hashing, and Python doesn't use this for `dict`.
- **Perfect hashing:** guarantees $O(1)$ worst case by designing a collision-free hash function for a known, fixed set of keys. Not practical for a general-purpose, mutable dictionary where keys are added dynamically.

For a general-purpose, dynamically-sized mapping, open addressing/chaining hash tables (what Python actually uses) remain the standard choice.

Edge Cases

- **Empty dictionary:** `{}.get("x")` returns `None` (or raises `KeyError` with `[]` access) — no crash, no scan needed since there's nothing to scan.
- **Duplicate keys during construction:** `{"a": 1, "a": 2}` silently keeps only the last value — dict keys are always unique.
- **Hash collisions:** two unrelated keys can hash to the same slot; correctness is preserved through probing/chaining, only performance is (minimally) affected.
- **Using an unhashable type as a key:** `{[1, 2]: "value"}` raises `TypeError: unhashable type: 'list'` immediately.
- **Very large dictionaries:** Python resizes the underlying table as it grows, keeping the average lookup cost flat even at scale.

Common Mistakes

1. **Assuming dict lookups are $O(1)$ worst case.** They're $O(1)$ *average case* — pathological collisions can degrade this, though it's rare in practice.
2. **Trying to use a list as a dictionary key.** Lists are mutable, so their hash could change after insertion, breaking the table. Python blocks this outright.
3. **Forgetting that dict key order matters for equality of custom objects.** If you override `__eq__` without also overriding `__hash__`, your object becomes unhashable or behaves unpredictably as a key.
4. **Confusing "dict is ordered" (insertion order, guaranteed since Python 3.7) with "dict is sorted."** Iteration order reflects insertion, not any hash-based ordering.
5. **Believing hashing eliminates all cost.** Computing a hash isn't free — for very large or complex keys, hashing itself has cost, though it's still dramatically cheaper than a full scan.

Interview Questions

- Why is dictionary lookup $O(1)$ on average, but not guaranteed $O(1)$ worst case?
- What's the difference between open addressing and chaining for collision resolution?
- Why must dictionary keys be hashable, and what does "hashable" actually require (`__hash__` and `__eq__`)?
- How does a hash table decide when to resize, and what does resizing cost?
- Could you implement a hash table using only a fixed-size array?

Similar Problems

- **Two Sum (LeetCode 1):** the classic use case for trading an $O(n^2)$ brute-force pair search for an $O(n)$ hash-map-based lookup.
- **Group Anagrams (LeetCode 49):** uses a hash map keyed by sorted characters to group items in $O(1)$ average insert/lookup per item.
- **Contains Duplicate (LeetCode 217):** directly tests whether you reach for a hash set instead of a nested loop.
- **LRU Cache (LeetCode 146):** combines a hash map with a doubly linked list to get $O(1)$ get/put — a natural next step after understanding basic hashing.

Key Takeaways

- List lookups are $O(n)$ because there's no relationship between a key's value and its storage position — you have to search.
- Dictionary lookups are $O(1)$ on average because a hash function computes the storage position directly from the key — you jump instead of searching.
- Collisions are handled through chaining or open addressing, and they add negligible average cost.
- Dictionary keys must be immutable (hashable) because a key's hash must never change after it's stored, or Python could never find it again.
- This same principle — trade a little upfront work for fast lookups later — underlies hash sets, database indexes, and caching systems everywhere.

Watch the Video

This breakdown pairs with the full video walkthrough, where you can see the list-scan and hash-jump compared side by side. Watch it here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series dedicated to opening up the black boxes developers rely on every day instead of just trusting them. Each episode picks one mechanism — a data structure, a language feature, a system design pattern — and breaks it down until the "magic" disappears and what's left is a clear, reusable mental model.

Call To Action

If this helped the mechanism finally click, the best way to support more breakdowns like it is simple: like the video on YouTube, subscribe to the channel, and subscribe to the Substack for deeper written companion pieces. Drop a comment with your guess on the concurrent-write question from the video — we'll cover it in a follow-up. And if you know another developer still repeating "dicts are fast" without knowing why, share this with them.

Quick Review

Why is a Python dict lookup $O(1)$ instead of $O(n)$ like a list?

It computes a hash of the key to jump straight to a slot, instead of scanning every item like a list search.

Time complexity of average-case dict get/set?

$O(1)$ average — hashing gives direct slot access; worst case degrades to $O(n)$ with heavy collisions.

Space complexity tradeoff dicts make for that speed?

$O(n)$ extra space for the hash table (slots), more memory than a plain list to enable constant-time lookup.

Trickiest gotcha: why can't a list be a dict key?

Keys must be immutable/hashable so their hash never changes; lists are mutable, so Python raises `TypeError`.

Dict lookup vs list lookup — key mental model difference?

List: check items one by one until found (linear scan). Dict: hash key, jump directly to its slot (near-instant).

Interview follow-up: what happens when two keys hash to the same slot?

A collision — Python probes for another open slot (open addressing), so worst-case lookup can degrade toward $O(n)$.

Why Python Dictionaries Are Faster Than You Think — free Python cheat sheet